

AI ASSISTED CODING

B.GEETHANJALI

2303A51005

Q1. Zero-Shot Prompting (Basic Lab Task)

Task: Write a Python function that classifies a given text as Spam or Not Spam using zero-shot prompting.

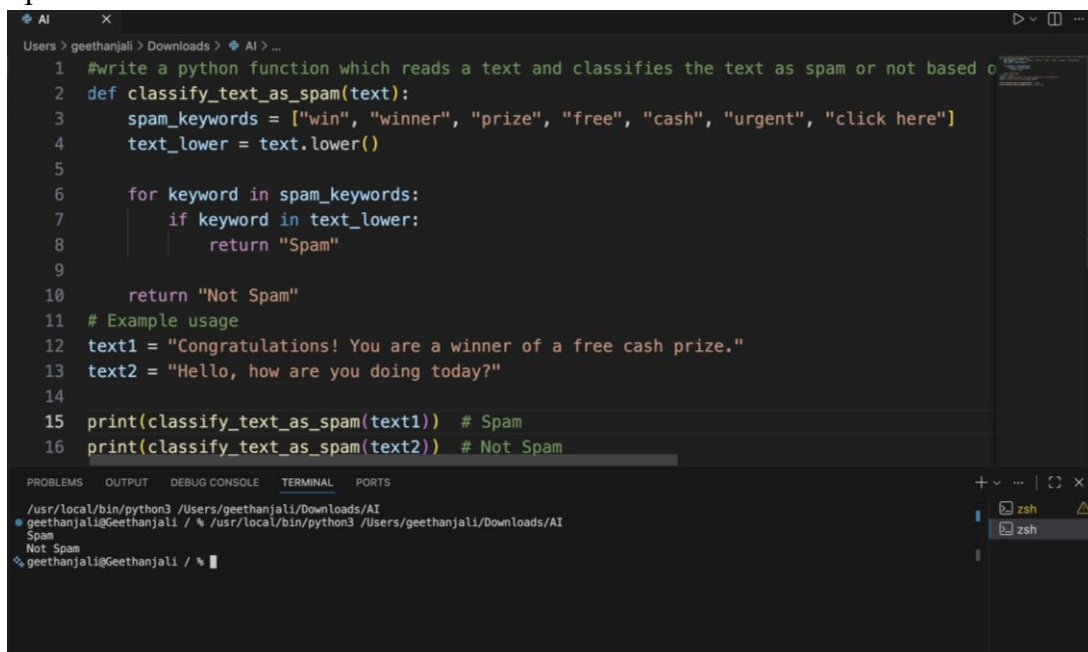
Steps:

1. Construct a prompt without any examples.
2. Clearly specify the output labels.
3. Display only the predicted label.

Input:

"Congratulations! You have won a free lottery ticket." Expected Output:

Spam



```
1 #write a python function which reads a text and classifies the text as spam or not based o
2 def classify_text_as_spam(text):
3     spam_keywords = ["win", "winner", "prize", "free", "cash", "urgent", "click here"]
4     text_lower = text.lower()
5
6     for keyword in spam_keywords:
7         if keyword in text_lower:
8             return "Spam"
9
10    return "Not Spam"
11 # Example usage
12 text1 = "Congratulations! You are a winner of a free cash prize."
13 text2 = "Hello, how are you doing today?"
14
15 print(classify_text_as_spam(text1)) # Spam
16 print(classify_text_as_spam(text2)) # Not Spam
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Spam
Not Spam
geethanjali@geethanjali / %
```

Q2. One-Shot Prompting (Emotion detection)

Task:

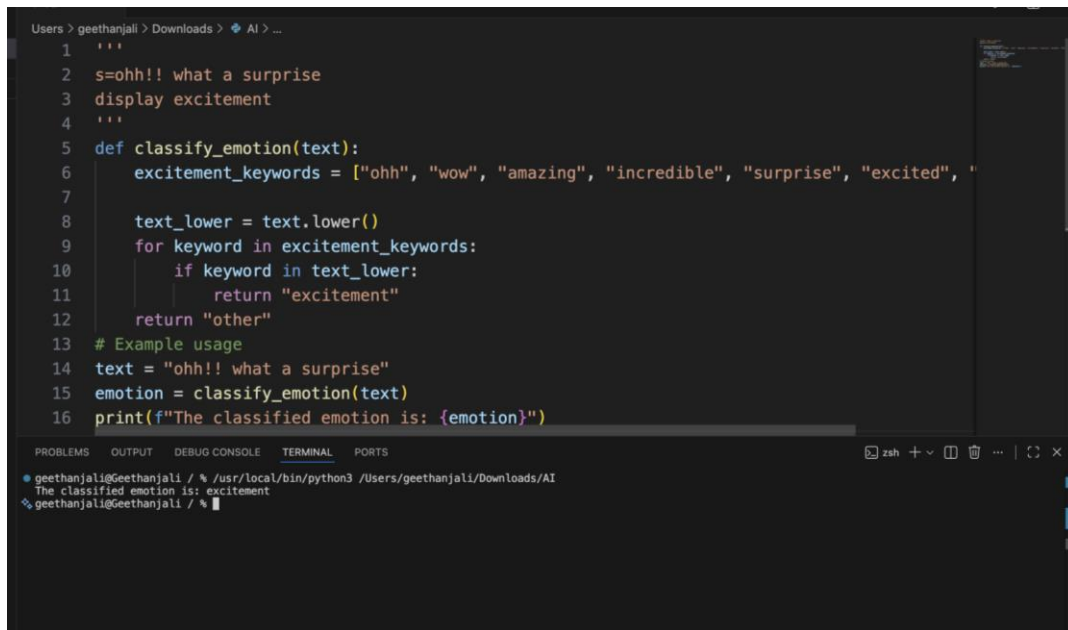
Write a Python program that detects the emotion of a sentence using one-shot prompting.

Emotions: ['happy', 'sad', 'angry', 'excited', 'nervous', 'neutral']

Steps:

1. Provide one labeled example inside the prompt.

2. Take a sentence as input.
3. Print the predicted emotion



```
1 '''
2 s=ohh!! what a surprise
3 display excitement
4 '''
5 def classify_emotion(text):
6     excitement_keywords = ["ohh", "wow", "amazing", "incredible", "surprise", "excited", "
7
8     text_lower = text.lower()
9     for keyword in excitement_keywords:
10         if keyword in text_lower:
11             return "excitement"
12     return "other"
13 # Example usage
14 text = "ohh!! what a surprise"
15 emotion = classify_emotion(text)
16 print(f"The classified emotion is: {emotion}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

geethanjali@geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
The classified emotion is: excitement
geethanjali@geethanjali / %

Q3. Few-Shot Prompting (Student Grading Based on Marks)

Task:

Write a Python program that predicts a student's grade based on marks using few-shot prompting.

Grades:

['A', 'B', 'C', 'D', 'F']

Grading Criteria (to be inferred from examples):

- 90–100 → A
- 80–89 → B
- 70–79 → C
- 60–69 → D
- Below 60 → F

```
Users > geethanjali > Downloads > AI > ...
1 '''
2 m=85
3 display B
4 m=55
5 display f
6 m=78
7 display c
8 m=61
9 display d
10 '''
11 def classify(m):
12     if m >= 80:
13         return "B"
14     elif m >= 70:
15         return "C"
16     elif m >= 60:
17         return "D"
18     else:
19         return "F"
20 marks = [85, 55, 78, 61]
21 for m in marks:
22     grade = classify(m)
23     print(f"m={m}\ndisplay {grade}")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
m=78
display C
m=61
display D
geethanjali@Geethanjali / %
```

Q4. Multi-Shot Prompting (Indian Zodiac Sign Prediction using Month Name)

Task:

Write a Python program that predicts a person's Indian Zodiac sign (Rashi) based on the month of birth (month name) using multi-shot prompting.

Indian Zodiac Order (Simplified Month-Based Model): The Indian Zodiac cycle starts in March with Mesha and follows this order: March → Mesha

April → Vrishabha

May → Mithuna

June → Karka

July → Simha

August → Kanya

September → Tula

October → Vrischika

November → Dhanu

December → Makara

January → Kumbha

February → Meen

```
Users > geethanjali > Downloads > AI > ...
1  '''
2  March
3  display Mesha
4  April
5  display Vrishabha
6  May
7  display Mithuna
8  June
9  display Karka
10 July
11 display Simha
12 August
13 display Kanya
14 September
15 display Tula
16 October
17 display Vrischika
18 November
19 display Dhanu
20 December
21 display Makara
22 January
23 display Kumbha
24 February
25 display Meena
26 '''
27 def get_zodiac_sign(month):
28     zodiac_signs = {
29         "March": "Mesha",
```

```
Users > geethanjali > Downloads > AI > ...
24 February
25 display Meena
26 '''
27 def get_zodiac_sign(month):
28     zodiac_signs = {
29         "March": "Mesha",
30         "April": "Vrishabha",
31         "May": "Mithuna",
32         "June": "Karka",
33         "July": "Simha",
34         "August": "Kanya",
35         "September": "Tula",
36         "October": "Vrischika",
37         "November": "Dhanu",
38         "December": "Makara",
39         "January": "Kumbha",
40         "February": "Meena"
41     }
42     return zodiac_signs.get(month, "Invalid month")# Example usage:
43 month = "April"
44 sign = get_zodiac_sign(month)
45 print(f"The zodiac sign for {month} is {sign}.")
46
```

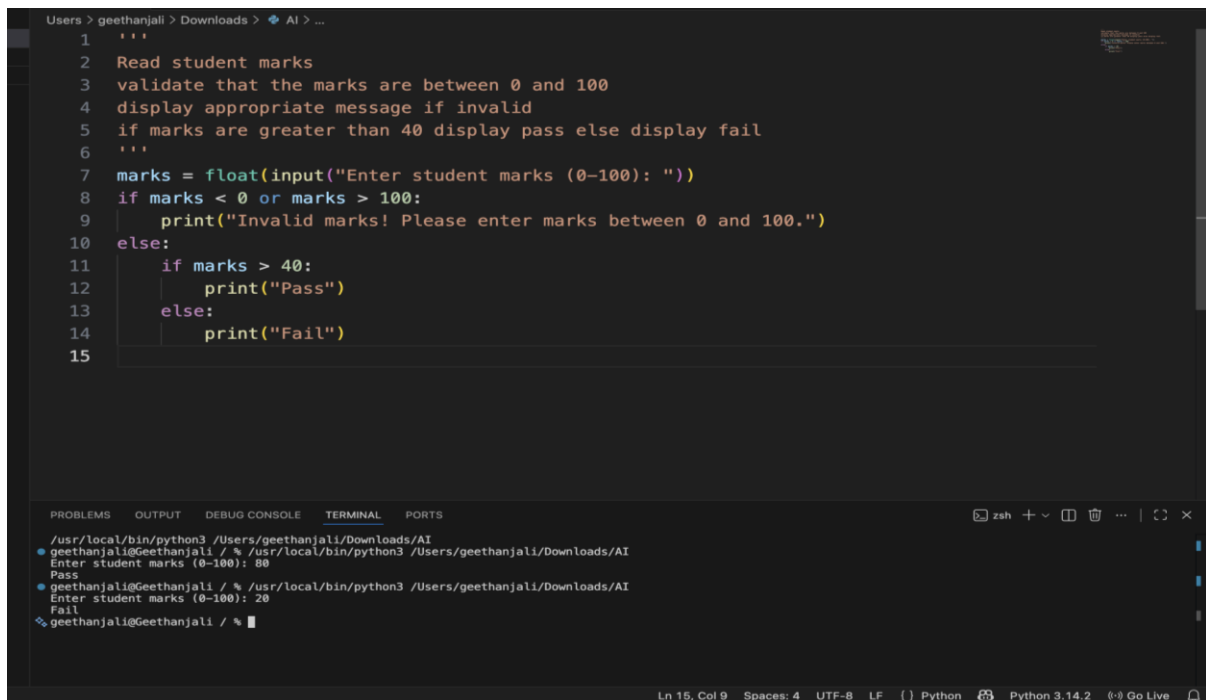
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
The zodiac sign for April is Vrishabha.
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
The zodiac sign for April is Vrishabha.
geethanjali@Geethanjali / %
```

Q5. Result Analysis Based on Marks

Task: Write a Python program that determines whether a student Passes or Fails based on marks using Chain-of-Thought (CoT) prompting.

Result Categories: ['Pass', 'Fail']



The image shows a VS Code editor window with a Python file open. The code is a simple program that reads student marks, validates them, and prints 'Pass' or 'Fail' based on a threshold of 40. The terminal at the bottom shows the program being executed twice: first with a mark of 80 resulting in 'Pass', and second with a mark of 20 resulting in 'Fail'.

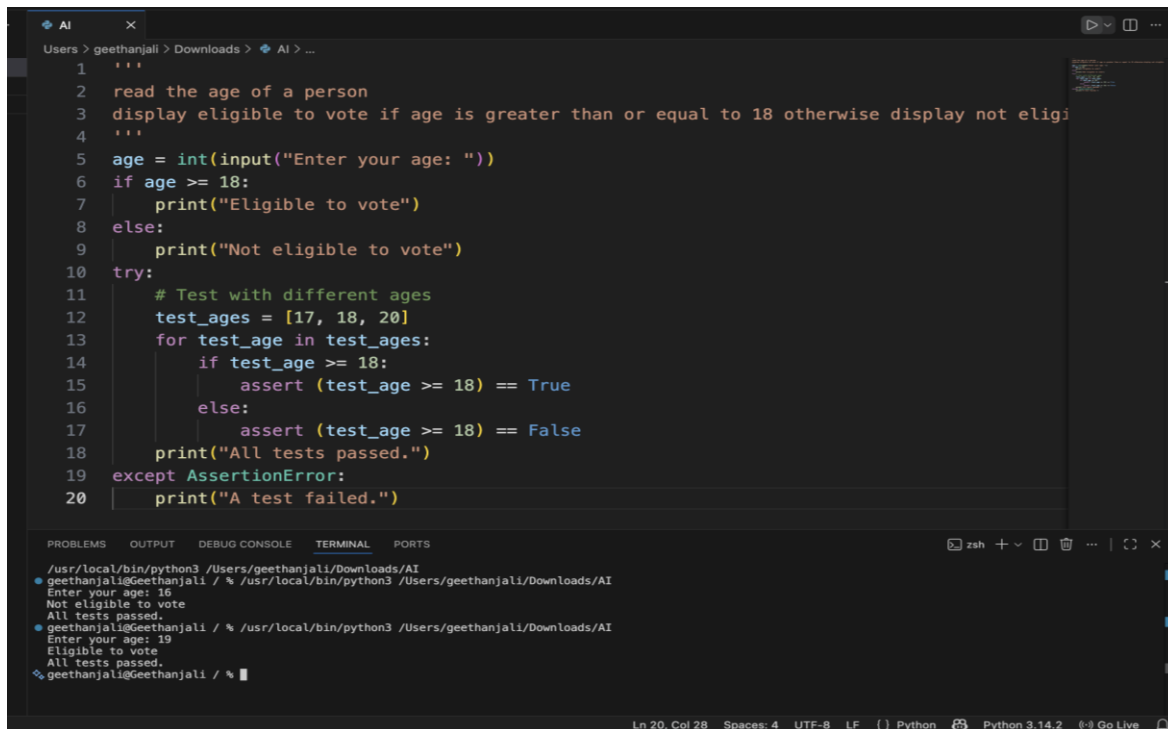
```
1 '''  
2 Read student marks  
3 validate that the marks are between 0 and 100  
4 display appropriate message if invalid  
5 if marks are greater than 40 display pass else display fail  
6 '''  
7 marks = float(input("Enter student marks (0-100): "))  
8 if marks < 0 or marks > 100:  
9     print("Invalid marks! Please enter marks between 0 and 100.")  
10 else:  
11     if marks > 40:  
12         print("Pass")  
13     else:  
14         print("Fail")  
15
```

Terminal Output:

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI  
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI  
Enter student marks (0-100): 80  
Pass  
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI  
Enter student marks (0-100): 20  
Fail  
geethanjali@Geethanjali / %
```

Q6 Voting Eligibility Check (Chain-of-Thought Prompting)

Task: Write a Python program that determines whether a person is eligible to vote using Chain-of-Thought (CoT) prompting.



```
1 '''
2 read the age of a person
3 display eligible to vote if age is greater than or equal to 18 otherwise display not eligi
4 '''
5 age = int(input("Enter your age: "))
6 if age >= 18:
7     print("Eligible to vote")
8 else:
9     print("Not eligible to vote")
10 try:
11     # Test with different ages
12     test_ages = [17, 18, 20]
13     for test_age in test_ages:
14         if test_age >= 18:
15             assert (test_age >= 18) == True
16         else:
17             assert (test_age >= 18) == False
18     print("All tests passed.")
19 except AssertionError:
20     print("A test failed.")
```

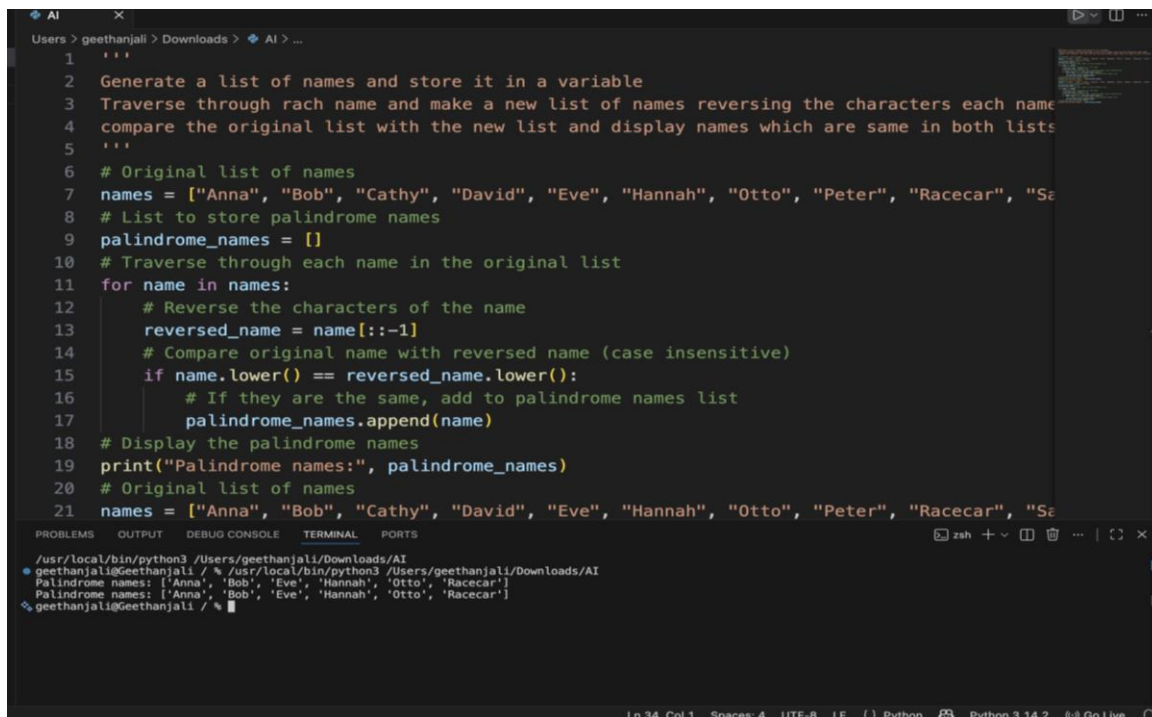
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Enter your age: 16
Not eligible to vote
All tests passed.
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Enter your age: 19
Eligible to vote
All tests passed.
geethanjali@Geethanjali / %

Ln 20, Col 28 Spaces: 4 UTF-8 LF () Python Python 3.14.2 Go Live

Q7 Prompt Chaining (String Processing – Palindrome Names)

Task: Write a Python program that uses the prompt chaining technique to identify palindrome names from a list of student names.



```
1 '''
2 Generate a list of names and store it in a variable
3 Traverse through each name and make a new list of names reversing the characters each name
4 compare the original list with the new list and display names which are same in both lists
5 '''
6 # Original list of names
7 names = ["Anna", "Bob", "Cathy", "David", "Eve", "Hannah", "Otto", "Peter", "Racecar", "Se
8 # List to store palindrome names
9 palindrome_names = []
10 # Traverse through each name in the original list
11 for name in names:
12     # Reverse the characters of the name
13     reversed_name = name[::-1]
14     # Compare original name with reversed name (case insensitive)
15     if name.lower() == reversed_name.lower():
16         # If they are the same, add to palindrome names list
17         palindrome_names.append(name)
18 # Display the palindrome names
19 print("Palindrome names:", palindrome_names)
20 # Original list of names
21 names = ["Anna", "Bob", "Cathy", "David", "Eve", "Hannah", "Otto", "Peter", "Racecar", "Se
```

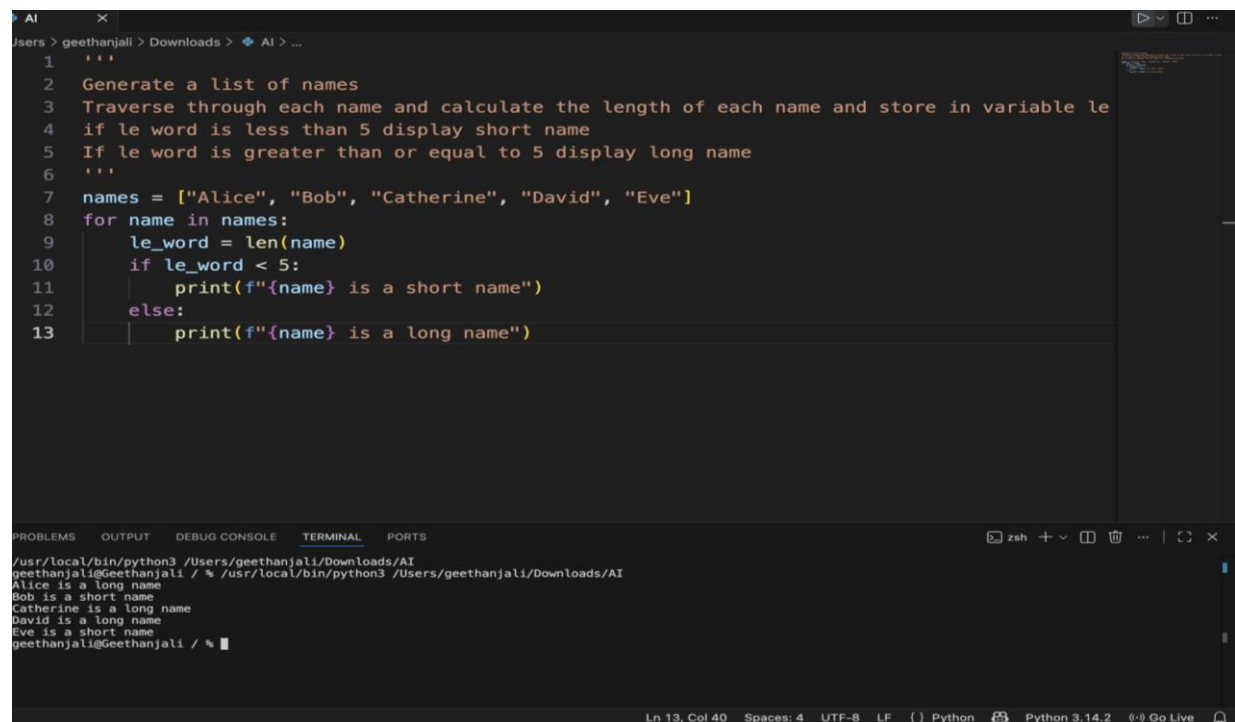
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Palindrome names: ['Anna', 'Bob', 'Eve', 'Hannah', 'Otto', 'Racecar']
geethanjali@Geethanjali / %

Ln 34, Col 1 Spaces: 4 UTF-8 LF () Python Python 3.14.2 Go Live

Q8 Prompt Chaining (String Processing – Word Length Analysis)

Task: Write a Python program that uses prompt chaining to analyze a list of words. In the first prompt, generate a list of words. In the second prompt, traverse the list and calculate the length of each word. In the third prompt, use the output of the previous step to determine whether each word is Short (length less than 5) or Long (length greater than or equal to 5), and display the result for each word



The image shows a VS Code editor window with a Python script in the main editor and its output in the terminal. The script is a Python program that generates a list of names, calculates their lengths, and prints whether they are short or long based on a length of 5.

```
1 '''
2 Generate a list of names
3 Traverse through each name and calculate the length of each name and store in variable le
4 if le word is less than 5 display short name
5 If le word is greater than or equal to 5 display long name
6 '''
7 names = ["Alice", "Bob", "Catherine", "David", "Eve"]
8 for name in names:
9     le_word = len(name)
10    if le_word < 5:
11        print(f"{name} is a short name")
12    else:
13        print(f"{name} is a long name")
```

The terminal output shows the execution of the script, displaying the results for each name:

```
/usr/local/bin/python3 /Users/geethanjali/Downloads/AI
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/AI
Alice is a long name
Bob is a short name
Catherine is a long name
David is a long name
Eve is a short name
geethanjali@Geethanjali / %
```

The status bar at the bottom indicates the current line and column (Ln 13, Col 40), the number of spaces (4), the encoding (UTF-8), the line ending (LF), the language (Python), and the Python version (Python 3.14.2).